MRI-RadAI: Backend MVP Build Brief

Explainable prostate lesion detection with a Claude vision model. Handoff document for an AI coding agent (Codex) and team members.

Version 1.0 · 10 November 2026 · Research prototype for a final-year project. Not a medical device.

> **Read this first.** This is a research and decision-support prototype. It must not be used for clinical decisions. Use only public, de-identified data (PROSTATEx) in the MVP. Every analysis output must show the disclaimer in Section 13. The landing page is already built and lives in `frontend/`. Do not rebuild it. The backend is the critical path.

Contents

1. Project summary and current status
2. MVP scope (in and out)
3. Backend tech stack
4. End-to-end flow
5. Repository layout
6. Configuration (environment variables)
7. Database schema
8. API contract
9. Image and DICOM pipeline
10. Claude vision integration (prompts, output schema, parsing)
11. Background processing
12. Error handling
13. Security, privacy and disclaimer rules
14. Testing and acceptance criteria
15. Frontend contract (for the app pages)
16. Build order for Codex
17. Phase 2 and later
18. Known risks and limitations
19. Decisions made in this version
20. Appendix A. Models and the ML training track

1. Project summary and current status

Goal. A radiologist uploads prostate MRI images (DICOM, or PNG/JPG exports). The backend sends selected slices from each sequence (T2-weighted, DWI, ADC) to a Claude vision model. The model returns structured JSON: whether suspicious regions are present, where they are (slice and normalized box), a confidence score, a review-priority level, and a plain-language reason. The backend validates and stores the result. The radiologist reviews it and records feedback (agree, disagree, adjust location, missed lesion).

Why a VLM. Unlike a black-box CNN, the model explains its reasoning in words. It needs no retraining to change its instructions, which makes it quick to iterate for a small project. The trade-off is that it is a general-purpose vision model, not a prostate-MRI model. Accuracy must be measured, not assumed (see Section 18).

Workstream	Status	Owner
Landing page (frontend)	Done	Frontend
Backend API, ingestion, Claude integration, database	To build (critical path)	Backend
App pages: upload, progress, results, feedback	To build after backend endpoints work	Frontend
Dataset, metrics, evaluation against ground truth	Parallel track	ML

2. MVP scope

In scope for the MVP

- Upload 1 to 3 images per analysis, each tagged with a sequence type (T2, DWI, ADC, or OTHER). Accept DICOM

(.dcm), PNG and JPG.
- Convert DICOM to windowed 8-bit grayscale PNG slices. Drop patient identifiers during ingestion.
- Run one Claude vision analysis per request over the selected slices.
- Parse and validate the JSON output. Store lesions, tokens and cost.
- Poll-based status updates for the frontend.
- Radiologist feedback endpoint.
- Swagger/OpenAPI docs, Docker Compose for local development, pytest suite.

Out of scope for the MVP (see Section 17)

- User accounts, login, institutions, roles. The MVP is single-tenant and unauthenticated, for local use only.
- Redis, Celery, and S3. The MVP uses local disk and in-process background tasks.
- 3D (Three.js) visualization. The MVP uses a 2D slice viewer with overlays.
- PDF report generation, PACS/DICOMweb, batch processing, multi-scan comparison.
- Serving the trained CNN baseline through the API. Training it is part of the ML track (Appendix A).

3. Backend tech stack

Component	Choice	Notes
Language	Python 3.11	Use the version pinned in the Dockerfile.
Web framework	FastAPI	Auto-generates OpenAPI docs at `/docs`.
ASGI server	Uvicorn	Run with `uvicorn app.main:app`.
Validation	Pydantic v2, pydantic-settings	Used for requests, responses, config, and the VLM output schema.
VLM client	`anthropic` Python SDK, `AsyncAnthropic`	The Python package is `anthropic`. The JavaScript package is not used here. Pin the version after install with `pip freeze`.
Database	PostgreSQL 16 (Docker)	Stores uploads, analyses, lesions, feedback, usage.
ORM and driver	SQLAlchemy 2.0 (async), psycopg 3	Connection string uses `postgresql+psycopg://`.
Migrations	Alembic	Initial migration creates the schema in Section 7.
Medical imaging	pydicom, numpy	Pixel data, rescale slope and intercept, windowing.
Image handling	Pillow	Grayscale conversion, resize, PNG encoding.
Uploads	python-multipart	Required by FastAPI for form and file uploads.
Testing	pytest, pytest-asyncio, httpx	Mock the Anthropic client in tests. Never call the live API in CI.
Lint and format	ruff, black	Run in CI and before each commit.
Containers	Docker, Docker Compose	Two services: `api` and `db`.
Phase 2	Redis 7 + Celery 5, boto3 (S3), prometheus-client, Sentry	Add only when the MVP works end to end.

Do not pin versions from memory. Install the latest compatible versions, run the tests, then freeze them into `requirements.txt`.

4. End-to-end flow

```
Frontend (upload page)
   |  POST /api/v1/uploads  (file + sequence_type)
   v
FastAPI: validate magic bytes and size -> store original temporarily
   |  DICOM: window + extract slices     PNG/JPG: normalize to grayscale
   v
Storage: derived PNG slices in STORAGE_DIR/uploads/{upload_id}/
   |  POST /api/v1/analyses  (upload_ids, optional slice_indices)
   v
Create analyses row (status=pending) -> return 202 with analysis_id
   |  BackgroundTask: status=processing
   v
VLM service: build message (labelled images + prompt) -> Claude API
   |  parse JSON -> validate (Pydantic) -> one repair retry if invalid
   v
Store lesions, tokens, cost, raw result -> status=completed (or failed)
   ^
   |  GET /api/v1/analyses/{id}  (frontend polls every 2 s)
Frontend results page -> POST /api/v1/analyses/{id}/feedback
```

5. Repository layout

```
prostate-lesion-ai/
  backend/
    app/
      main.py              FastAPI app, CORS, router registration, startup checks
      config.py            pydantic-settings Settings (all env vars in Section 6)
      db.py                async engine, session factory, declarative Base
      models.py            SQLAlchemy ORM models (Section 7)
      schemas.py           Pydantic request/response models and VlmResult schema
      api/
        health.py          GET /health
        uploads.py         POST/GET uploads, GET preview
        analyses.py        POST/GET analyses, GET list
        feedback.py        POST feedback
      services/
        storage.py         local-disk storage behind a small interface (swap to S3 later)
        dicom_io.py        read DICOM/PNG/JPG, windowing, slice selection, PNG encoding
        prompts.py         SYSTEM_PROMPT, build_user_text(), PROMPT_VERSION
        vlm.py             Anthropic client wrapper, message building, usage accounting
        parsing.py         extract JSON from model text, validate, build repair prompt
        analysis_runner.py run_analysis(analysis_id): full pipeline, status updates
      utils/
        logging.py         JSON logs; never logs image bytes, filenames or raw output
    migrations/            Alembic
    tests/
      conftest.py          creates synthetic PNG and synthetic DICOM in a temp dir
      test_dicom_io.py
      test_parsing.py
      test_uploads_api.py
      test_analyses_api.py mocked Anthropic client
    requirements.txt
    Dockerfile
    .env.example
    README.md
  frontend/                already built (landing page); app pages are added later
  ml/                      dataset loader, metrics (ML track)
  docker-compose.yml       api + db, at repo root
  README.md
```

6. Configuration (environment variables)

Variable	Default	Purpose
`APP_ENV`	`dev`	`dev`, `test`, or `prod`. Prod refuses to start without auth (Phase 2).
`ANTHROPIC_API_KEY`	none	Required for analyses. If missing, `POST /analyses` returns 503 and `/health` reports `vlm: "not_configured"`.
`CLAUDE_MODEL`	`claude-sonnet-5-5`	Model ID. Verify the current ID at docs.claude.com before use. Never hard-code it.
`CLAUDE_MAX_TOKENS`	`2048`	Output token cap per call.
`CLAUDE_TIMEOUT_SECONDS`	`120`	Per-request timeout.
`CLAUDE_MAX_RETRIES`	`3`	Passed to the SDK client (handles 429 and 5xx with backoff).

`DATABASE_URL`	`postgresql+psycopg://mri:mri@db:5432/mri`	Database connection.
`STORAGE_DIR`	`./storage`	Local folder for derived slices and temporary files.
`MAX_UPLOAD_MB`	`50`	Per file.
`SLICES_PER_SEQUENCE`	`5`	Default slices sent per sequence when the client does not specify.
`MAX_IMAGES_PER_ANALYSIS`	`12`	Hard cap on images in one Claude call.
`IMAGE_MAX_SIDE_PX`	`1024`	Longest side after resizing. Lower it to cut token cost.
`KEEP_ORIGINAL_DICOM`	`false`	If false, the original DICOM is deleted after slices are extracted.
`PRICE_INPUT_PER_MTOK`, `PRICE_OUTPUT_PER_MTOK`	none	USD per million tokens, copied from Anthropic's pricing page. Cost is computed from these values. Do not hard-code prices.
`CORS_ORIGINS`	`http://localhost:5173,http://localhost:3000`	Comma-separated. Match the landing page's dev port.

7. Database schema

Use UUID primary keys everywhere. Timestamps are UTC. No user, institution, or patient-identity tables in the MVP.

uploads

Column	Type	Notes
id	uuid, pk	
original_filename	text	Stored for display only. Never used as a file path.
sha256	char(64), indexed	Same file plus same sequence_type returns the existing upload.
sequence_type	enum T2, DWI, ADC, OTHER	Set by the client.
source_format	enum dicom, png, jpeg	Detected from magic bytes, not the extension.
size_bytes	int	
slice_count	int	Number of derived slices.
width, height	int	Of the derived slices.
created_at	timestamptz	

analyses

Column	Type	Notes
id	uuid, pk	
status	enum pending, processing, completed, failed	
model	text	Model ID actually used.
prompt_version	text	From `prompts.PROMPT_VERSION`.
selected_slices	jsonb	List of {upload_id, slice_index, sequence_type}.
lesions_detected	bool, nullable	
overall_risk_level	enum low, medium, high, nullable	
overall_confidence	int 0-100, nullable	
image_quality	jsonb, nullable	From model output.
summary	text, nullable	
raw_result	jsonb, nullable	Validated model output. Never returned in logs.
input_tokens, output_tokens	int, nullable	From the API `usage` field, summed over repair

		calls.
cost_usd	numeric(10,4), nullable	Computed from configured prices.
processing_ms	int, nullable	
error_message	text, nullable	Generic message only. No stack traces or PHI.
created_at, started_at, completed_at	timestamptz	

analysis_uploads

Join table: `analysis_id` and `upload_id` form the primary key.

lesions

Column	Type	Notes
id	uuid, pk	
analysis_id	uuid, fk, on delete cascade	
position	int	Order within the analysis (1-based).
upload_id, slice_index	uuid, int	Mapped back by the backend from the model's image_index.
sequence_type	enum	
bbox_x, bbox_y, bbox_w, bbox_h	float	Normalized 0 to 1, relative to the slice image.
anatomical_region	enum apex, mid, base, unknown	
confidence	int 0-100	
risk_level	enum low, medium, high	
characteristics	text[]	For example: "restricted diffusion", "low T2 signal".
explanation	text	Max 300 characters.

feedback

Column	Type	Notes
id	uuid, pk	
analysis_id	uuid, fk, cascade	
lesion_id	uuid, fk, nullable, set null	Null for whole-analysis feedback.
feedback_type	enum agree, disagree, adjust_location, missed_lesion	
notes	text, max 2000 chars	
corrected_bbox	jsonb, nullable	{x, y, w, h}, normalized. Required when feedback_type is adjust_location.
created_at	timestamptz	

8. API contract

Base path `/api/v1`. All JSON. Errors use the shape `{"error": {"code": "...", "message": "..."}}`. Every analysis response includes the disclaimer field.

Health

```
GET /health
200 -> {"status": "ok", "db": "ok", "vlm": "configured" | "not_configured"}
```

Uploads

```
POST /api/v1/uploads          multipart/form-data
  file:               .dcm, .png, .jpg, .jpeg (checked by magic bytes)
  sequence_type: "T2" | "DWI" | "ADC" | "OTHER"
201 -> {
  "upload_id": "uuid",
  "sequence_type": "T2",
  "source_format": "dicom",
  "slice_count": 40,
  "width": 384, "height": 384,
  "status": "ready"
}
200 -> same body if the sha256 + sequence_type already exists (deduplicated)
413 file too large | 415 unsupported type | 422 unreadable DICOM or no pixel data

GET /api/v1/uploads/{upload_id}                 -> metadata (same shape as above)
GET /api/v1/uploads/{upload_id}/slices/{index}  -> image/png (derived slice, 0-based index)
```

Analyses

```
POST /api/v1/analyses
  {
    "upload_ids": ["uuid", "uuid"],          1 to 3 uploads, distinct sequence_types
    "slice_indices": {"": [10, 14, 18]}   optional; default = evenly spaced
  }
202 -> {"analysis_id": "uuid", "status": "pending", "selected_images": 6}
400 invalid request | 404 unknown upload | 409 total images > MAX_IMAGES_PER_ANALYSIS
503 ANTHROPIC_API_KEY not configured

GET /api/v1/analyses/{analysis_id}
200 -> {
  "analysis_id": "uuid",
  "status": "completed",
  "lesions_detected": true,
  "overall_risk_level": "medium",
  "overall_confidence": 71,
  "summary": "...",
  "image_quality": {"rating": "adequate", "limitations": ["mild motion on DWI"]},
  "lesions": [
    {
      "id": "uuid",
      "position": 1,
      "upload_id": "uuid",
      "sequence_type": "ADC",
      "slice_index": 14,
      "bbox": {"x": 0.42, "y": 0.55, "w": 0.12, "h": 0.10},
      "anatomical_region": "mid",
      "confidence": 74,
      "risk_level": "medium",
      "characteristics": ["low ADC", "restricted diffusion"],
      "explanation": "..."
    }
  ],
  "model": "claude-sonnet-5-5",
  "prompt_version": "v1.0",
  "usage": {"input_tokens": 9120, "output_tokens": 640, "cost_usd": 0.0432},
  "processing_ms": 48210,
  "disclaimer": "Research prototype. Not for clinical use. Requires radiologist review."
}
200 with status "pending" | "processing" | "failed" (failed includes "error_message")

GET /api/v1/analyses?limit=20&offset=0         -> list, newest first (summary fields only)
```

Feedback

```
POST /api/v1/analyses/{analysis_id}/feedback
  {
    "feedback_type": "agree" | "disagree" | "adjust_location" | "missed_lesion",
    "lesion_id": "uuid",                optional
    "notes": "string, max 2000",        optional
    "corrected_bbox": {"x":0.4,"y":0.5,"w":0.1,"h":0.1}   required for adjust_location
  }
201 -> {"feedback_id": "uuid", "status": "saved"}
409 analysis not completed | 422 validation error
```

9. Image and DICOM pipeline

1. **Type detection.** Read the first bytes. DICOM files have the marker `DICM` at byte offset 128. PNG starts with `\x89PNG`. JPEG starts with `FF D8 FF`. Anything else returns 415, even if the extension looks right.
2. **Storage name.** Save the original under a server-generated path, for example `STORAGE_DIR/tmp/{uuid}.bin`. Never use the client's filename in a path.
3. **DICOM read.** `pydicom.dcmread(path)`. If pixel data is missing or cannot be decoded, return 422. Multi-frame objects

are treated as a slice stack. Apply `RescaleSlope` and `RescaleIntercept` when present.

4. **Windowing.** Use `WindowCenter` and `WindowWidth` if present. Otherwise use the 1st and 99th percentiles of each volume. Map to uint8 0 to 255.
5. **Normalize to grayscale.** Convert RGB to L. Resize so the longest side is at most `IMAGE_MAX_SIDE_PX` with LANCZOS. Keep the aspect ratio.
6. **Write derived slices.** Save PNGs as `STORAGE_DIR/uploads/{upload_id}/slice_{i:03d}.png`. Record `slice_count`, `width`, and `height`.
7. **Strip identifiers.** Do not save DICOM header fields such as PatientName, PatientID, or dates of birth. Delete the original DICOM unless `KEEP_ORIGINAL_DICOM=true`.
8. **Slice selection.** If the client gives `slice_indices`, use them (validate range). Otherwise pick `SLICES_PER_SEQUENCE` evenly spaced indices across the middle 60 percent of the stack. For PNG and JPG, the single image is the only slice. This is a placeholder heuristic. The ML track should replace it with prostate-localization once available.

10. Claude vision integration

10.1 Call structure

- One Claude call per analysis. The message contains the system prompt, and a user turn that alternates a short text label and an image for each selected slice.
- Label format: `Image 3 of 6. Sequence: ADC. Slice index 14 of 40.` The model refers to images by number only. Filenames and IDs are never sent.
- Images are PNG, grayscale, base64-encoded. Use `client.messages.create` via `AsyncAnthropic`, with `model=CLAUDE_MODEL`, `max_tokens=CLAUDE_MAX_TOKENS`, and `temperature=0`.
- Record `response.usage.input_tokens` and `output_tokens`. Add up usage across any repair call.

10.2 System prompt (PROMPT_VERSION = "v1.0")

```
You are a research assistant that helps a radiologist review prostate MRI images.
You are NOT a diagnostic device. You do not diagnose cancer or state a Gleason grade.
You describe imaging features that are visible in the provided images and estimate how
suspicious each region looks, using PI-RADS-style terms (T2-weighted signal, diffusion
restriction on DWI, low signal on ADC).

Rules:
- Only describe what is visible. Do not invent measurements, patient details, or findings.
- If an image is non-diagnostic (motion, artifact, poor contrast, wrong anatomy), say so
  and lower your confidence.
- Be conservative. When in doubt, flag a region for radiologist review rather than dismiss it.
- Confidence is your estimate (0-100) that a region is a real suspicious finding in the
  images provided. It is not a probability of cancer.
- Output ONLY one JSON object that matches the schema in the user message. No prose
  before or after it, no markdown code fences.
```

10.3 User text (sent after the images)

```
Review the images above (T2, DWI, ADC slices of the prostate). Return findings as JSON.

Coordinates: bbox values are normalized from 0 to 1 relative to the image as shown.
(x, y) is the top-left corner. Keep the box tight around the region.
risk_level means review priority: low, medium, or high.
Return at most 5 lesions. If no suspicious region is visible, set lesions_detected to
false and return an empty lesions list.

Schema:
{
  "lesions_detected": boolean,
  "overall_risk_level": "low" | "medium" | "high",
  "overall_confidence": integer 0-100,
  "image_quality": {
    "rating": "good" | "adequate" | "poor",
    "limitations": [string]
  },
  "lesions": [
    {
      "image_index": integer (the image number from its label),
      "bbox": {"x": number, "y": number, "w": number, "h": number},
      "anatomical_region": "apex" | "mid" | "base" | "unknown",
      "confidence": integer 0-100,
      "risk_level": "low" | "medium" | "high",
      "characteristics": [string],
      "explanation": string (max 300 characters)
    }
  ],
  "summary": string (max 500 characters)
}
```

10.4 Parsing and validation (parsing.py)

1. Strip any code fences. Take the substring from the first `{` to its matching `}`. Call `json.loads`.
2. Validate with a Pydantic model `VlmResult`. Enforce: confidence 0 to 100; bbox values 0 to 1 with `x + w ≤ 1` and `y + h ≤ 1`; `image_index` within the number of images sent; at most 5 lesions; `lesions_detected` consistent with the lesions list.
3. If parsing or validation fails, send one repair turn. Include the previous assistant text and the validation error, and ask for corrected JSON only. Allow one repair per analysis.
4. If the repair also fails, set status to `failed`, store the raw text in `raw_result` for debugging, and set a generic error message.
5. Map `image_index` back to `upload_id` and `slice_index` on the server. Store a lesion row per valid lesion.

10.5 Prompt versioning

Every analysis stores `prompt_version` and `model`. Keep prompts in `prompts.py` as constants. Changing a prompt means bumping the version. This is needed to compare runs and report results honestly later.

11. Background processing

- **MVP:** `POST /analyses` creates the row with status `pending`, then schedules `run_analysis(analysis_id)` with FastAPI `BackgroundTasks`. The function opens its own database session.
- **Idempotency:** if an analysis is already `completed`, `run_analysis` does nothing. If it is `processing` when a duplicate arrives, do nothing.
- **Timing:** status changes to `processing` at start, then to `completed` or `failed`. Record `processing_ms`.
- **Known limit:** a server restart loses in-flight analyses. Such rows stay `processing`. On startup, mark any `processing` rows older than 15 minutes as `failed` with "interrupted".
- **Phase 2:** move `run_analysis` to a Celery task (Redis broker). Keep the same function signature so the switch is mechanical.

12. Error handling

Situation	Response or behavior
Wrong file type (magic bytes)	415 at upload
File over MAX_UPLOAD_MB	413 at upload (stream and stop reading at the limit)
DICOM unreadable or no pixel data	422 at upload. Message says what is missing, without patient data.
Missing API key	503 on `POST /analyses`. Startup logs a warning.
Anthropic 429 or 5xx	SDK retries (CLAUDE_MAX_RETRIES). If still failing, analysis becomes `failed` with "model service unavailable".
Anthropic timeout	Analysis becomes `failed` with "model request timed out". Allow the user to submit again.
Output invalid after one repair	Analysis `failed` with "model output could not be validated". Raw text kept in DB only.
Database down at startup	Health reports `db: "error"`. Requests return 503.
Any unhandled exception	500 with generic message. Full traceback goes to server logs only, without image data or patient fields.

13. Security, privacy and disclaimer rules

- **Data.** The MVP uses only public, de-identified data such as PROSTATEx. No real patient images in development or demos.
- **Production gate.** Real patient data requires all of the following first: a Business Associate Agreement (or equivalent) with the model provider, institutional approval, HIPAA-appropriate hosting, encryption at rest and in transit, authentication and role-based access, audit logging, and a retention policy. None of these exist in the MVP. The MVP must not be described as HIPAA-compliant.
- **Logging.** Log IDs, status changes, durations, token counts, and HTTP status codes. Never log image bytes, client filenames, DICOM header values, or raw model output at INFO level.

- **Input safety.** Validate by magic bytes. Generate storage names on the server. Reject path characters in any client-supplied value that becomes part of a path.
- **Secrets.** API keys only from environment variables. `.env` is in `.gitignore`. `.env.example` holds placeholders only.
- **Disclaimer.** Every analysis response includes: "Research prototype. Not for clinical use. Requires radiologist review." The frontend shows the same text on the results page and in any export.
- **Model output.** Never present a model output as a diagnosis. Use terms such as "suspicious region for review" and "estimated confidence".
- **Rate limiting.** Not in the MVP. Add in Phase 2 before any shared deployment.

14. Testing and acceptance criteria

14.1 Tests

Area	What to test
dicom_io	Synthetic DICOM (created in conftest with pydicom) windows to uint8. Slope and intercept applied. Missing pixel data returns the right error. Slice selection is evenly spaced and in range. PNG and JPG are converted to grayscale and resized.
parsing	Clean JSON. JSON wrapped in prose. JSON in code fences. Truncated JSON. Out-of-range bbox. Image index out of range. lesions_detected mismatch. Repair path reached and succeeds. Repair fails.
uploads API	PNG and DICOM accepted. .txt with a .png name rejected (415). Oversized file rejected (413). Duplicate upload returns 200.
analyses API	With the Anthropic client mocked: 202 on create, status moves to completed, lesion rows and tokens stored, cost computed from configured prices. Missing key returns 503. Failure path stores generic error.
feedback API	Saves each feedback type. adjust_location without corrected_bbox returns 422. Feedback on a pending analysis returns 409.

CI must never call the live API. The mock returns fixture JSON.

14.2 Manual live check (once per prompt version)

Run 3 PROSTATEx cases (one per sequence where available) through the live API. Record latency, tokens, cost, and whether the output validated first time. Keep the results in `ml/results/`. Do not claim accuracy from these three cases.

14.3 MVP acceptance criteria

1. `docker compose up` starts `api` and `db`. `GET /health` returns 200 with `db: "ok"`.
2. Swagger at `/docs` lists every endpoint in Section 8 with schemas.
3. A PNG upload and a DICOM upload both succeed. A renamed text file is rejected.
4. An analysis with 3 sequences and 5 slices each completes in under 3 minutes on the default model (measured, not assumed).
5. The result matches the schema. Every bbox is within 0 to 1. Every lesion maps to a real upload and slice.
6. Input tokens, output tokens, and cost are stored and returned.
7. The disclaimer appears in every analysis response.
8. Feedback is saved and visible in the database.
9. `pytest` passes with no network access. Service-layer coverage is at least 70 percent.
10. Setup from a clean clone works using only `README.md` and `.env.example`.

15. Frontend contract (for the app pages)

The landing page is done. Match its stack and styling (check `frontend/package.json`). Do not add new dependencies just for this. Build these pages after the backend endpoints in Section 8 respond:

- **Upload page.** Up to 3 files. Each file needs a sequence type (T2, DWI, ADC, OTHER). Show the 413, 415, and 422 messages from the API. On success, call `POST /analyses` and go to the progress page.
- **Progress page.** Poll `GET /analyses/{id}` every 2 seconds. Stop on `completed` or `failed`. Show the error message when failed, with a retry option that creates a new analysis.
- **Results page.** Show a 2D slice viewer with a dropdown for sequence and slice. Draw each lesion's bbox as an overlay (use the normalized coordinates scaled to the displayed image). List lesions with confidence, risk level, characteristics, and explanation. Show the disclaimer at the top. Show usage and processing time in a small footer.
- **Feedback.** Buttons per lesion (agree, disagree, adjust location) and for the whole analysis (missed lesion). Adjust

location lets the user drag a box to a new position and sends `corrected_bbox`.

- **History page (optional).** Uses `GET /analyses`.
- **Not in MVP.** Three.js or any 3D view. Also no PDF export.

16. Build order for Codex

Work in this order. Finish and test each step before starting the next.

1. Scaffold `backend/` with the layout in Section 5. Add `config.py`, `db.py`, `/health`. Run it.
2. Add models, Alembic initial migration, and the Docker Compose file with Postgres.
3. Implement `dicom_io.py` and the uploads endpoints. Write its tests with synthetic files. No Claude calls yet.
4. Implement `prompts.py`, `parsing.py`, and `vlm.py`. Test parsing with fixtures. Test the client with a mock.
5. Implement `analysis_runner.py` and the analyses endpoints with BackgroundTasks. Test end to end with the mock.
6. Run one live analysis with a PROSTATEx case. Record tokens, latency, and cost. Tune `IMAGE_MAX_SIDE_PX` and slice count if needed.
7. Implement feedback and the list endpoint.
8. Write `README.md`, `.env.example`, and check the acceptance criteria in Section 14.3.
9. Hand the working API to the frontend team for the app pages in Section 15.

17. Phase 2 and later

- Authentication (JWT), users, institutions, roles. Required before any shared use.
- Redis and Celery for background work. S3 for storage. Signed URLs for downloads.
- Multi-slice volume viewer, then Three.js 3D overlays for the results page.
- PDF report (reportlab or WeasyPrint) with findings, images, and the disclaimer.
- Evaluation endpoint and dashboard: sensitivity, specificity, AUC-ROC, F1, Dice, against radiologist annotations from the ML track.
- Prompt A/B testing with stored prompt versions. Compare against a fine-tuned CNN baseline.
- Feedback loop: export feedback as a dataset for the ML track. Do not retrain automatically.
- DICOMweb (QIDO-RS, WADO-RS) integration with a PACS, only after the compliance gate in Section 13 is met.
- Rate limiting, Prometheus metrics, structured logging to ELK, Kubernetes manifests.

18. Known risks and limitations

Risk	Implication and mitigation
General-purpose model, not validated on prostate MRI	Expect weak sensitivity and specificity at first. Measure against PROSTATEx annotations before making any accuracy claim. Report results with their limits.
2D slices lose 3D context	Lesions may be missed or double-counted across slices. Treat results as candidate regions for review.
Bounding boxes are approximate	Measure overlap (IoU) against ground-truth masks in the ML track. Do not report millimetre coordinates from 2D images.
Output variability	temperature=0 reduces but does not remove variation. Repeat runs on the same case to measure stability.
Model IDs and prices change	Model and prices are configuration. Check them before each deployment and record them with each analysis.
Token cost per analysis	Many images per call is the main cost driver. Measure with the live check in Section 14.2 before choosing defaults.
Hallucinated findings	Validation catches format errors, not false content. Radiologist review and the disclaimer are the safeguards. Prompt rules are not a substitute.
In-process background tasks	Analyses are lost on restart. Acceptable for the MVP only.
Dataset licensing	Check PROSTATEx terms on The Cancer Imaging Archive before publishing results or redistributing data.

19. Decisions made in this version

Topic	Earlier draft	This brief	Why
Lesion location	Millimetre coordinates (x_mm, y_mm, z_mm)	Slice index plus normalized 2D box	A single 2D image cannot give reliable millimetre positions. Claiming them would be false precision.
Model	claude-3-5-sonnet-20241022	`CLAUDE_MODEL` config, default `claude-sonnet-5-5`	The old ID is outdated. Verify the ID before each deployment.
Python SDK	Mixed references to an npm-style package name	`anthropic` package, `AsyncAnthropic`	The Python SDK is named `anthropic`.
Version pins	Specific versions listed from memory	Install latest compatible, then freeze	Pins from memory were unverified and may not install together.
Cost figures	$0.10 to $0.50 per scan	Measured in the live check, using configured prices	The earlier figure was an estimate and is not sourced.
Prompt flow	Four prompts, multi-turn	One structured call plus one repair turn	Fewer calls, lower cost, simpler to test. Multi-turn can return in Phase 2.
Async work	Celery and Redis from day one	BackgroundTasks in MVP	Fewer moving parts. Same function moves to Celery later.
Users and institutions	Tables in the MVP	Deferred to Phase 2	Not needed for a single-user prototype.
Compliance	Described as HIPAA-compliant	Production gate in Section 13	Compliance depends on contracts, hosting, and controls that do not exist yet.
Frontend 3D	Three.js in the MVP	2D viewer in MVP, 3D in Phase 2	Keeps the MVP achievable. The landing page already covers the 3D story.

Appendix A. Models and the ML training track

Short answer. The team does not train Claude. The MVP calls Claude through the API with a fixed, versioned prompt, so the only lever on the VLM is the prompt. The one model the team trains is a small supervised CNN baseline. It answers a comparison question: how does a conventional trained model perform on the same test lesions as the VLM? Recommended baseline: a 2D ResNet-18 with ImageNet weights, fine-tuned on lesion patches.

A.1 Which model for what

Role	Model	Why
VLM (MVP, all analyses)	`claude-sonnet-5-5`	Good balance of image reasoning and cost. This is the default in `CLAUDE_MODEL`.
VLM comparison run	`claude-opus-5-5`	Higher-capability model for a one-off comparison on the test set. Report its cost separately because it is more expensive.
VLM prompt iteration	Same model as production	Do not iterate on a cheaper model and assume the results transfer. Behaviour differs between models.
Trained baseline (ML track)	ResNet-18, ImageNet-pretrained (torchvision)	Small enough to train on a free GPU in under an hour. Strong default for a dataset of this size.
Later baseline, if ResNet-18 is weak	3D ResNet-18 with MedicalNet pretraining (MONAI loaders)	Uses full volume context. More compute, more setup. Only if time allows.

Model IDs come from the current Anthropic model list at the time of writing. Check docs.claude.com before each run and record the exact ID with the results.

A.2 Baseline training design

Item	Decision
Labels	PROSTATEx provides lesion coordinates and a clinical-significance (ClinSig) label per lesion. Use ClinSig as a binary target: clinically significant vs not.
Input	2D patch centred on each lesion coordinate, from the same slice, stacked as channels: T2, ADC, and DWI. Start with about 64 by 64 pixels and tune. Confirm the patch size against the resampled pixel spacing.
Negatives	Patches sampled from prostate tissue away from any labelled lesion. Keep the positive to negative ratio fixed across splits.
Architecture	ResNet-18. Replace the first conv to accept 3 channels (or use the 3 sequences as RGB-like channels, then fine-

	tune). Replace the final layer with one output logit.
Loss and optimizer	Binary cross-entropy with a positive class weight. AdamW, learning rate 1e-4, batch size 64, up to 30 epochs.
Early stopping	On validation AUC-ROC, patience 5 epochs. Keep the best checkpoint only.
Augmentation	Horizontal flips and small rotations (about ±10 degrees). Avoid intensity changes large enough to alter ADC meaning.
Splits	Patient-level 60/20/20, stratified by ClinSig. A patient never appears in two splits. Tune only on validation. Touch the test set once per model version.
Repeats	Three random seeds. Report mean and standard deviation for every metric.
Framework and hardware	PyTorch and torchvision. A free Colab or Kaggle GPU is enough. No local GPU needed.
Tracking	MLflow (already in the plan). Log config, seeds, metrics, and the checkpoint hash. Store the YAML config beside the weights.

A.3 Why not the other options

- **Segmentation model (nnU-Net and similar):** PROSTATEx gives lesion points and labels, not segmentation masks. A segmentation model would need new annotation work. Revisit only if masks become available.
- **Training from scratch:** a dataset of a few hundred patients will overfit. Always start from ImageNet or medical pretrained weights.
- **Fine-tuning Claude:** not part of this project. The VLM is evaluated through prompts only.
- **Dice coefficient:** the earlier metrics list included it. Dice needs masks, so it does not apply to point labels. Use the point-in-box rate below instead.

A.4 Evaluation and fair comparison with the VLM

- **Test set:** the same held-out patients and lesions for both the CNN and the VLM. Never give the VLM a set the CNN did not see, or the reverse.
- **Metrics:** AUC-ROC, sensitivity and specificity at a threshold chosen on the validation set, and the false-positive rate per patient.
- **Point-in-box rate for the VLM:** the share of annotated lesions whose annotated point falls inside a predicted box on the correct slice. This fits point labels and replaces IoU for this dataset.
- **Uncertainty:** bootstrap confidence intervals over patients. A small test set can flip rankings by chance, so report intervals next to every number.
- **Prompt tuning:** iterate prompts on the validation set. Score each prompt version once on the test set.

A.5 Outputs of the ML track

- Trained weights and config in `ml/models/`. Keep weights out of git (Git LFS or external storage). Keep the config and metrics in git.
- A short results note in `ml/results/`: dataset version, split seed, model versions, metrics with intervals, and the VLM comparison.
- The CNN is not served by the MVP API. Serving it is a Phase 2 item, behind the same response schema.

Prepared for team handoff. Update `PROMPT_VERSION`, the model ID, and prices before each new run. Record the results of every live check in `ml/results/`.